Глава 1. Теоретическая часть

1.1. Персистентные структуры данных: постановка задачи

Большинство классических структур данных являются эфемерными. Это значит, что при каждом изменении предыдущее состояние структуры теряется и работать можно только с текущей версией. Во многих задачах этого недостаточно, так как нужно уметь вернуться к одному из прошлых состояний, сравнить несколько состояний между собой или продолжить работу со старого состояния, не потеряв при этом новое.

Структура данных называется персистентной (от англ. persistent – устойчивый, долговременный), если она сохраняет все свои предыдущие версии и позволяет обращаться к ним после выполнения изменений. Понятие персистентности и общие методы построения таких структур описаны в работе Дж. Дрисколла, Н. Сарнака, Д. Слейтора и Р. Тарьяна 1989 года. В зависимости от того, что разрешено делать со старыми версиями, различают несколько видов персистентности:

- частичная персистентность – все версии доступны для чтения, но изменять можно только последнюю, поэтому версии идут друг за другом в виде цепочки;
- полная персистентность – любую версию можно читать и изменять, при изменении старой версии появляется новая ветка, и версии образуют дерево;
- конфлюэнтная персистентность – кроме этого можно объединять две версии в одну (например, при конкатенации), и версии образуют ориентированный ациклический граф;
- функциональная персистентность – узлы структуры после создания не изменяются совсем. Так устроены структуры данных в функциональных языках программирования (Haskell, OCaml, Clojure).

Сформулируем задачу формально. Пусть над структурой выполняется последовательность операций, и после i-й операции получается версия v_i. Нужно реализовать структуру, которая:

1. по номеру версии v позволяет выполнить любую операцию чтения над состоянием v;
2. по номеру версии v позволяет выполнить операцию изменения и получить новую версию v', не изменив v;
3. тратит на каждую операцию время и память, сравнимые с затратами эфемерной структуры.

Самое простое решение – копировать всю структуру перед каждым изменением. Оно выполняет первые два требования, но не выполняет третье, потому что при n элементах каждая операция требует $\Theta(n)$ времени и памяти. В данной работе рассматривается персистентный стек. Для него третье требование выполняется полностью, и все операции над любой версией выполняются за $O(1)$.

1.2. Стек как абстрактный тип данных

Стек – это абстрактный тип данных, который хранит последовательность элементов и работает по принципу LIFO (Last In, First Out – «последним пришел, первым ушел»). Добавлять и удалять элементы можно только с одного конца, который называется вершиной стека. Стек поддерживает следующие операции:

Операция	**Описание**	**Эфемерный стек**

Операция	Описание	Эфемерный стек
push(x)	положить элемент x на вершину	O(1) (амортизированно для массива)
pop()	удалить элемент с вершины	O(1)
top()	вернуть элемент на вершине, не удаляя его	O(1)
size() / empty()	число элементов / проверка на пустоту	O(1)

Обычный (эфемерный) стек чаще всего реализуют на динамическом массиве или на односвязном списке. В обоих случаях операция pop удаляет информацию о верхнем элементе: ячейка массива перезаписывается, а узел списка освобождается. Поэтому вернуться к состоянию до pop без отдельного журнала операций нельзя.

В персистентном стеке у каждой операции есть дополнительный аргумент – номер версии, к которой она применяется. Операции изменения возвращают номер новой версии:

```
push(v, x) -> v'     // новая версия: стек версии v с элементом x на вершине
pop(v)     -> v'     // новая версия: стек версии v без верхнего элемента
top(v)     -> x      // элемент на вершине версии v
size(v)    -> k      // число элементов в версии v
```

Версия v при этом остается неизменной и доступной для дальнейших операций.

1.3. Структура персистентного стека

Основная идея состоит в том, что односвязный список, узлы которого не изменяются после создания, уже является полностью персистентным стеком. Операция push не меняет существующие узлы, а только создает новый узел со ссылкой на старую вершину. Операция pop вообще не создает и не изменяет узлы, новой версией просто становится ссылка на следующий узел. В результате разные версии используют общие узлы, и копировать ничего не нужно.

Персистентный стек состоит из двух частей.

Первая часть – узлы. Каждый узел представляет собой неизменяемую запись (value, next, size), где value – хранимый элемент, next – ссылка на узел, лежащий под ним (для дна стека null), size – количество элементов в стеке, вершиной которого является этот узел. Значение size вычисляется один раз при создании узла по формуле size = size(next) + 1.

Вторая часть – массив версий roots, в котором для каждой версии хранится ссылка на ее вершину. Версия 0 соответствует пустому стеку (roots[0] = null), а номер новой версии равен следующему свободному индексу в массиве roots.

```
class Node:
    value, next, size          # поля не изменяются после создания

roots = [null]                 # версия 0 – пустой стек

push(v, x):
    node = Node(value = x, next = roots[v], size = size(v) + 1)
    roots.append(node)
    вернуть len(roots) - 1      # номер новой версии
```

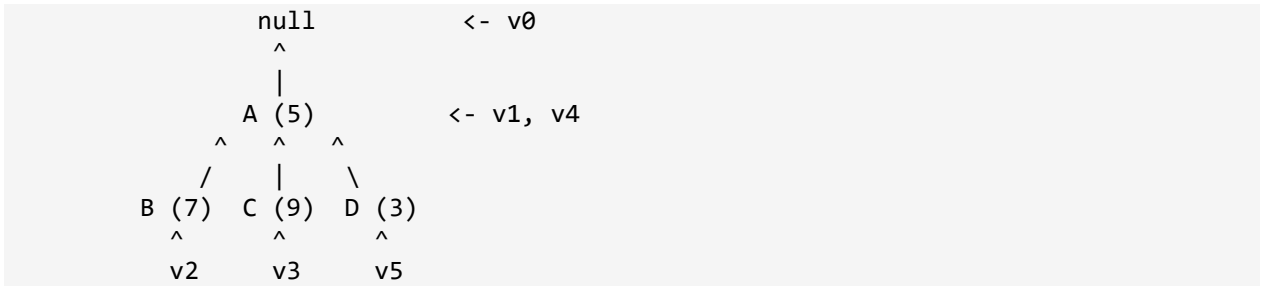
```
pop(v):
    если roots[v] == null: ошибка «стек пуст»
    roots.append(roots[v].next) # новый узел НЕ создается
    вернуть len(roots) - 1

top(v):  вернуть roots[v].value
size(v): вернуть 0, если roots[v] == null, иначе roots[v].size
```

Рассмотрим пример. Выполним следующие операции (справа указано содержимое версии от вершины к дну):

```
v0 = пустой стек                    -> []
v1 = push(v0, 5)                    -> [5]
v2 = push(v1, 7)                    -> [7, 5]
v3 = push(v1, 9)    // ветвление    -> [9, 5]
v4 = pop(v2)                        -> [5]
v5 = push(v4, 3)                    -> [3, 5]
```

В итоге будет создано всего четыре узла A(5), B(7), C(9) и D(3), причем узел A используется всеми непустыми версиями. Связи между узлами образуют дерево, корнем которого является пустой стек (null):



Содержимое любой версии можно получить, пройдя от ее вершины до корня дерева. Версии v1 и v4 совпадают и указывают на один и тот же узел. Версии v2, v3 и v5 получены из одного состояния [5], но при этом ни один элемент не копировался.

1.4. Свойства и инварианты

Правильная работа персистентного стека основана на следующих инвариантах.

1. Узлы не изменяются. После создания узла его поля value, next и size больше не меняются, поэтому операция над одной версией не может повлиять на другую версию.
2. Ссылки направлены на более старые узлы. Поле next нового узла указывает на узел, созданный раньше него, или на null. Значит, в графе узлов нет циклов, и он является деревом с корнем null (деревом версий).
3. Версии соответствуют путям в дереве. Содержимое версии v от вершины к дну совпадает с последовательностью значений на пути от roots[v] до корня.
4. Размер хранится правильно. Для любого узла u выполняется size(u) = size(u.next) + 1 и size(null) = 0, то есть size(u) равно глубине узла u в дереве версий.
5. Массив версий только растет. Элементы массива roots добавляются в конец и не перезаписываются, поэтому номер версии, который получил пользователь, остается правильным до конца работы программы.

Утверждение. Для любой последовательности операций и любой версии v значения top(v), size(v) и содержимое версии совпадают с тем, что получилось бы в обычном стеке после выполнения той же цепочки операций, которая привела к версии v.

Доказательство проведем индукцией по номеру версии. База: версия 0 является пустым стеком, roots[0] = null, и путь до корня пуст. Шаг: пусть утверждение верно для всех версий с номерами меньше k. Если версия k получена операцией push(v, x), где v < k, то ее путь состоит из нового узла со значением x и пути версии v. По предположению индукции путь v описывает правильный стек, значит путь k описывает тот же стек с элементом x на вершине, как и должно быть после push. Если версия k получена операцией pop(v), то ее путь совпадает с путем версии v без первого узла, то есть это стек версии v без вершины. Так как по инварианту 1 узлы версии v не менялись, операция над версией v не затронула другие версии, что и требовалось доказать.

1.5. Общие методы обеспечения персистентности

Чтобы понять, чем персистентный стек отличается от других решений, рассмотрим общие методы, с помощью которых можно сделать персистентной любую связную структуру данных.

- Полное копирование. Перед каждым изменением вся структура копируется. Метод подходит для любой структуры, но требует $\Theta(n)$ времени и памяти на каждую операцию.
- Копирование пути (path copying). Копируются только изменяемые узлы и узлы на пути от них до корня, остальные узлы остаются общими для разных версий. Для сбалансированных деревьев это дает $O(\log n)$ на операцию.
- Метод «толстых» узлов (fat node). В каждом поле узла хранится история его значений с номерами версий. Изменение требует $O(1)$ памяти, но чтобы прочитать поле в нужной версии, приходится искать по истории за $O(\log m)$, где m – количество версий.
- Копирование узлов (node copying). Этот метод предложен Дрисколлом и соавторами и объединяет два предыдущих. Для структур, в которых на каждый узел ссылается ограниченное число других узлов, он дает амортизированно $O(1)$ времени и памяти на изменение.

Для стека копирование пути сводится к самому простому случаю. Изменяется только вершина, а путь от измененного узла до корня состоит из одной этой вершины. Поэтому персистентный стек имеет такие же асимптотические оценки, как и обычный стек, без дополнительных затрат.

1.6. Персистентная очередь

В теме курсовой работы указан стек или очередь, поэтому кратко рассмотрим, почему персистентная очередь устроена сложнее. Очередь работает по принципу FIFO: элементы добавляются в конец, а извлекаются из начала. Изменения происходят на обоих концах последовательности, и способ с общим хвостом, который используется в стеке, здесь уже не подходит.

Обычную очередь часто реализуют на двух стеках (входном и выходном), и тогда операции выполняются за амортизированное $O(1)$. Но для персистентной структуры амортизированная оценка не работает, так как дорогую операцию (перекладывание всего входного стека в выходной) можно много раз применить к одной и той же старой версии, и каждый раз она будет стоить $\Theta(n)$. Чтобы получить $O(1)$ в худшем случае, нужны более сложные схемы, например очередь Худа и Мелвилла или очереди К. Окасаки с ленивыми вычислениями.

Есть и более простой метод со сложностью O(log n), который использует то же дерево узлов. Все добавленные элементы хранятся в дереве так же, как в персистентном стеке (новый узел ссылается на предыдущий конец очереди), а версия очереди задается парой (tail, len), где tail – узел последнего элемента, len – длина очереди. Тогда push добавляет узел за O(1), pop уменьшает len на единицу за O(1), а первый элемент очереди находится в предке узла tail на len - 1 уровней выше. Такого предка можно найти методом двоичного подъема (binary lifting) за O(log n), если хранить в каждом узле ссылки на предков на расстоянии 2^j:

$$up_j(u) = up_{j-1}\left(up_{j-1}(u)\right), \quad up_0(u) = u.next$$

Для этого нужно O(log n) дополнительной памяти на каждый узел. В данной работе реализуется персистентный стек, так как все его операции выполняются за O(1), а реализация получается простой и наглядной. Персистентная очередь может рассматриваться как возможное развитие работы.

1.7. Асимптотический анализ

Для оценки сложности будем использовать стандартные асимптотические обозначения. Для функций f(n), g(n) ≥ 0:

- f(n) = O(g(n)), если существуют c > 0 и n_0, такие что f(n) ≤ c·g(n) при всех n ≥ n_0 (оценка сверху);
- f(n) = Ω(g(n)), если существуют c > 0 и n_0, такие что f(n) ≥ c·g(n) при всех n ≥ n_0 (оценка снизу);
- f(n) = Θ(g(n)), если одновременно f(n) = O(g(n)) и f(n) = Ω(g(n)) (точная оценка).

Введем обозначения: n – количество элементов в версии, к которой применяется операция, m – общее количество выполненных операций (количество версий), k – количество элементов, которые нужно вывести.

Сначала оценим время выполнения операций. Каждая из операций push, pop, top и size выполняет постоянное число действий: обращение к массиву roots по индексу, создание не более одного узла и чтение полей. Это число не зависит ни от n, ни от m, поэтому каждая операция выполняется за Θ(1) в худшем случае, а не только в среднем. Так как любая операция требует хотя бы Ω(1) времени, лучше этих оценок получить нельзя.

Операция	**Время**	**Доп. память**	**Комментарий**
push(v, x)	Θ(1)	Θ(1)	один новый узел + элемент roots
pop(v)	Θ(1)	Θ(1)	только элемент roots, узел не создается
top(v), size(v)	Θ(1)	–	чтение полей узла-вершины
доступ к любой версии	Θ(1)	–	индекс в массиве roots
вывод содержимого версии	Θ(k)	–	обход пути до корня
m произвольных операций	Θ(m)	Θ(m)	суммарно для всех версий

Теперь оценим затраты памяти. Каждая операция push создает один узел, а каждая операция любого вида добавляет одну ссылку в массив roots. Поэтому после m операций общий объем памяти равен Θ(m), и при этом доступны все m + 1 версий.

Сравним эти оценки с простым копированием. Если перед каждой операцией копировать стек целиком, то i-я операция над стеком размера n_i стоит $\Theta(n_i)$. В худшем случае, когда выполняется m операций push подряд, общие затраты времени и памяти равны

$$\sum_{i=1}^{m} i = \frac{m(m+1)}{2} = \Theta(m^2)$$

а у персистентного стека Θ(m). Например, при $m = 10^5$ при копировании придется хранить около $5 \cdot 10^9$ элементов, что не поместится в оперативную память, а персистентному стеку нужно примерно 10^5 узлов.

Другой способ возвращаться к прошлым состояниям – хранить обычный стек и журнал обратных операций (undo-log). Откат на d шагов назад стоит Θ(d), а чтобы перейти к версии из другой ветки, нужно откатиться до общего предка и заново выполнить операции, на что может уйти до Θ(m) времени. Кроме того, при таком подходе нельзя читать несколько версий одновременно. В персистентном стеке доступ к любой версии выполняется за Θ(1).

Отдельно рассмотрим высоту дерева версий. Глубина узла равна размеру соответствующего стека, поэтому высота дерева не больше максимального размера стека. На сложность операций она не влияет, так как ни одна операция не спускается по дереву больше чем на один уровень. Обход дерева нужен только для вывода содержимого версии, который занимает Θ(k).

Также нужно сказать об освобождении памяти. Если на узел не ссылается ни одна версия и ни один другой узел, он становится недостижимым. В языках со сборкой мусора (Python, Java) такие узлы удаляются автоматически, а в C++ для этого можно использовать подсчет ссылок (std::shared_ptr). В курсовой работе все версии хранятся до завершения программы, поэтому узлы не удаляются.

1.8. Сравнение с альтернативными структурами

Структура	**push / pop**	**Доступ к старой версии**	**Память на операцию**	**Достоинства**	**Недостатки**
Эфемерный стек (массив)	O(1)*	невозможен	O(1)	простота, высокая скорость	старые версии теряются
Эфемерный стек + журнал отката	O(1)	от O(d) до O(m)	O(1)	малый расход памяти	нет одновременного доступа к версиям, медленное переключение веток
Полное копирование	O(n)	O(1)	O(n)	универсальность	квадратичный расход памяти
Персистентный	O(log n)	O(1)	O(log n)	доступ к любому	сложнее,

Структура	push / pop	Доступ к старой версии	Память на операцию	Достоинства	Недостатки
массив / дерево отрезков				элементу по индексу	медленнее в log n раз
«Толстые» узлы (fat node)	O(log m)	O(log m)	O(1)	общий метод для любых структур	поиск по истории версий
Персистентный стек (список)	O(1)	O(1)	O(1)	оптимальные оценки, простота	доступ только к вершине; k-й элемент – за O(k)

* – амортизированная оценка с учетом расширения динамического массива.

Из таблицы видно, что если нужны только операции стека (добавление и удаление с вершины), то лучше всего подходит персистентный стек на неизменяемом списке. По асимптотике он не хуже обычного стека ни в одной операции и при этом сохраняет все версии. Если же нужен доступ к элементам по индексу, лучше использовать персистентный массив или персистентное дерево отрезков, но тогда операции замедляются в O(log n) раз.

1.9. Примеры задач, решаемых с помощью выбранной структуры

1. Отмена и повтор действий (undo/redo). В текстовых и графических редакторах после каждого действия появляется новая версия состояния, и вернуться к любому моменту истории или начать от него новую ветку можно за O(1). По такому принципу работает дерево отмены (undo tree) в редакторах Vim и Emacs.
2. Системы контроля версий. В Git каждый коммит ссылается на родительский коммит и не изменяется после создания, а ветки являются указателями на последние коммиты, примерно как версии в персистентном стеке.
3. Перебор с возвратом. В алгоритмах поиска с возвратом (backtracking), при обходе игровых деревьев и в интерпретаторах языка Prolog персистентный стек позволяет хранить текущий путь решения и не копировать его при каждом ветвлении.
4. Интерпретаторы функциональных языков. В Lisp, Haskell, OCaml и Clojure неизменяемые списки являются основной структурой данных, а замыкания используют общие части окружения.
5. Олимпиадные задачи на откаты и дерево версий:
    - Codeforces 707D «Persistent Bookcase» (https://codeforces.com/problemset/problem/707/D). Нужно выполнять операции над состоянием и возвращаться к версии после k-й операции. Задача решается построением дерева версий, которое устроено так же, как дерево узлов персистентного стека;
    - Codeforces 1858E1 «Rollbacks (Easy Version)» (https://codeforces.com/problemset/problem/1858/E1). Есть массив и операции «добавить в конец», «удалить k последних элементов» и «отменить последнее изменение», то есть по сути стековые операции с откатом версий;
    - Codeforces 1858E2 «Rollbacks (Hard Version)» (https://codeforces.com/problemset/problem/1858/E2). Та же задача, но запросы нужно обрабатывать в онлайн-режиме, поэтому хранить версии приходится персистентно.

На сайтах acmp.ru и informatics.msk.ru задач именно на персистентный стек нет. Вместо них можно взять задачи из раздела «Стек» на informatics.msk.ru (реализация стека, проверка правильности скобочной последовательности) и добавить к ним требование хранить и восстанавливать предыдущие состояния стека.

Источники к главе 1

1. Driscoll J. R., Sarnak N., Sleator D. D., Tarjan R. E. Making data structures persistent // Journal of Computer and System Sciences. – 1989. – Vol. 38, № 1. – P. 86–124.
2. Okasaki C. Purely Functional Data Structures. – Cambridge University Press, 1998.
3. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. – 3-е изд. – М.: Вильямс, 2013.
4. Hood R., Melville R. Real-time queue operations in pure LISP // Information Processing Letters. – 1981. – Vol. 13, № 2. – P. 50–54.
5. Кнут Д. Э. Искусство программирования. Т. 1. Основные алгоритмы. – 3-е изд. – М.: Вильямс, 2000.
6. Персистентный стек // Викиконспекты ИТМО: neerc.ifmo.ru/wiki.
7. Demaine E. Advanced Data Structures. Lecture 1: Persistent Data Structures // MIT OpenCourseWare 6.851: ocw.mit.edu.